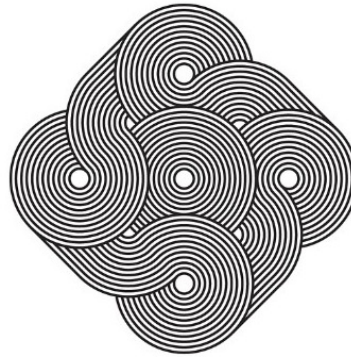




Design Patterns and SOLID Principles

Comprehensive Documentation

SE3140: System Design and Modeling

Faculty of Information and Communication
Technologies

Fall 2025

Lecturer: Engr. Tekoh Palma

Presentation Date: 7th January 2026

Group Members

1. Kamdeu Yamdjeuson Neil Marshall [ICTU20241386]
2. Nju Leslie chegue []
3. Atsimbon Joshua []
4. Fru Chi Ehud Neba []

Contents

1	Introduction	2
2	Design Patterns Overview	2
2.1	What Are Design Patterns?	2
2.2	Difference Between Design Patterns and Algorithms	3
2.3	Types of Design Patterns	3
3	Creational Design Patterns	4
3.1	Singleton Pattern	4
3.2	Factory Method Pattern	5
4	Structural Design Patterns	6
4.1	Adapter Pattern	7
4.2	Decorator Pattern	8
5	Behavioral Design Patterns	10
5.1	Observer Pattern	10
6	SOLID Principles	12
6.1	Single Responsibility Principle (SRP)	13
6.2	Open/Closed Principle (OCP)	15
6.3	Liskov Substitution Principle (LSP)	18
7	Conclusion	20

Key Concepts:

- Design Patterns
- SOLID Principles
- Software Architecture
- Maintainability
- Reusability

1 Introduction

Software systems inevitably grow in complexity as requirements evolve. Without proper design approaches, software becomes difficult to maintain, extend, and reuse. **Design Patterns** and **SOLID Principles** provide proven solutions to recurring software design problems and help developers create robust, flexible, and maintainable systems.

This document provides a comprehensive explanation of **Design Patterns** and **SOLID Principles**. Each design pattern is explained using:

- **Intent** - The purpose of the pattern
- **Motivation** - Why the pattern is needed
- **Structure** - UML or class relationships
- **Code Examples** - Practical implementations

Each SOLID principle is demonstrated with **clear, well-commented code examples**.

***Note:** Design patterns are not silver bullets. They should be applied judiciously based on the specific context and requirements of your project.*

2 Design Patterns Overview

2.1 What Are Design Patterns?

Design patterns are **general reusable solutions** to commonly occurring problems in software design. They are not finished designs that can be directly converted into code; instead, they are **templates or guidelines** that describe how to solve a problem in different situations.

Characteristic	Description
Best Practices	Represent solutions refined over time through collective experience
Readability	Improve code readability and maintainability
Reusability	Promote reusability and scalability of code
Language Independence	Can be implemented in any object-oriented language
Problem-Solving	Address specific design problems, not algorithmic problems

Table 1: Key Characteristics of Design Patterns

2.2 Difference Between Design Patterns and Algorithms

Aspect	Design Patterns	Algorithms
Purpose	Solve software design problems	Solve computational processing problems
Level	High-level system design	Low-level step-by-step logic
Output	Structure and relationships between classes	A specific result or computation
Reusability	Conceptual and architectural	Code-oriented and mathematical
Example	Singleton , Factory , Observer	Sorting, Searching, Graph traversal

Table 2: Design Patterns vs. Algorithms

2.3 Types of Design Patterns

Design patterns are classified into three main categories:

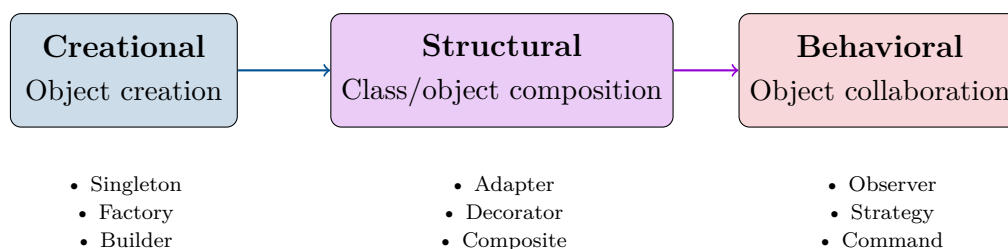


Figure 1: Three Categories of Design Patterns

3 Creational Design Patterns

Creational patterns abstract the instantiation process. They help make a system independent of how its objects are created, composed, and represented.

3.1 Singleton Pattern

Intent: Ensure a class has only one instance and provide a global point of access to it.

Motivation: Singleton is useful when exactly one object is needed to coordinate actions across the system (e.g., configuration manager, logging service).

Structure: **Private** static instance, **private** constructor, **public** static access method.

```
1 #include <iostream>
2
3 class Singleton {
4 private:
5     // 1. Private static instance
6     static Singleton* instance;
7
8     // 2. Private constructor
9     Singleton() {
10         std::cout << "Singleton instance created." << std::endl;
11     }
12
13     // Prevent copying
14     Singleton(const Singleton&) = delete;
15     Singleton& operator=(const Singleton&) = delete;
16
17 public:
18     // 3. Public static access method
19     static Singleton* getInstance() {
20         if (instance == nullptr) {
21             instance = new Singleton();
22         }
23         return instance;
24     }
25
26     void doSomething() {
27         std::cout << "Singleton is doing something." << std::endl;
28     }
29 };
30
31 // Initialize static member
32 Singleton* Singleton::instance = nullptr;
33
34 // Usage example
```

```
35 int main() {
36     // Get the singleton instance
37     Singleton* s1 = Singleton::getInstance();
38     s1->doSomething();
39
40     // This will return the same instance
41     Singleton* s2 = Singleton::getInstance();
42
43     // s1 and s2 are the same instance
44     std::cout << "Are they same? " << (s1 == s2 ? "Yes" : "No") << std
45     ::endl;
46
47     return 0;
48 }
```

Listing 1: Singleton Pattern Implementation in C++

3.2 Factory Method Pattern

Intent: Define an interface for creating objects but allow subclasses to alter the type of objects created.

Motivation: Factory Method promotes loose coupling by eliminating the need to bind application-specific classes into the code.

Structure: Product interface → ConcreteProduct → Creator (factory) class.

```
1 // Product interface
2 interface Shape {
3     void draw();
4 }
5
6 // Concrete Products
7 class Circle implements Shape {
8     @Override
9     public void draw() {
10         System.out.println("Drawing a Circle");
11     }
12 }
13
14 class Rectangle implements Shape {
15     @Override
16     public void draw() {
17         System.out.println("Drawing a Rectangle");
18     }
19 }
20
21 class Square implements Shape {
22     @Override
23     public void draw() {
24         System.out.println("Drawing a Square");
25     }
26 }
```

```
25     }
26 }
27
28 // Creator (Factory) class
29 class ShapeFactory {
30     // Factory method
31     public Shape getShape(String shapeType) {
32         if (shapeType == null) {
33             return null;
34         }
35
36         if (shapeType.equalsIgnoreCase("CIRCLE")) {
37             return new Circle();
38         } else if (shapeType.equalsIgnoreCase("RECTANGLE")) {
39             return new Rectangle();
40         } else if (shapeType.equalsIgnoreCase("SQUARE")) {
41             return new Square();
42         }
43
44         return null;
45     }
46 }
47
48 // Client code
49 public class FactoryPatternDemo {
50     public static void main(String[] args) {
51         ShapeFactory shapeFactory = new ShapeFactory();
52
53         // Get objects without knowing the concrete classes
54         Shape shape1 = shapeFactory.getShape("CIRCLE");
55         shape1.draw();
56
57         Shape shape2 = shapeFactory.getShape("RECTANGLE");
58         shape2.draw();
59
60         Shape shape3 = shapeFactory.getShape("SQUARE");
61         shape3.draw();
62     }
63 }
```

Listing 2: Factory Method Pattern Implementation in Java

4 Structural Design Patterns

Structural patterns are concerned with how classes and objects are composed to form larger structures.

4.1 Adapter Pattern

Intent: Convert the interface of a class into another interface that clients expect.

Motivation: Adapter allows reuse of existing classes whose interfaces do not match the system requirements.

Structure: Target interface $\leftarrow$ Adapter $\leftarrow$ Adaptee.

```
1 #include <iostream>
2 #include <string>
3
4 // Legacy class with incompatible interface
5 class LegacyRectangle {
6 private:
7     int x1, y1, x2, y2;
8
9 public:
10    LegacyRectangle(int x1, int y1, int x2, int y2)
11        : x1(x1), y1(y1), x2(x2), y2(y2) {
12        std::cout << "LegacyRectangle: " << x1 << "," << y1
13                << " to " << x2 << "," << y2 << std::endl;
14    }
15
16    void oldDraw() {
17        std::cout << "LegacyRectangle: oldDraw()." << std::endl;
18    }
19 };
20
21 // New interface expected by client
22 class Rectangle {
23 public:
24     virtual void draw(int x, int y, int width, int height) = 0;
25     virtual ~Rectangle() {}
26 };
27
28 // Adapter class
29 class RectangleAdapter : public Rectangle {
30 private:
31     LegacyRectangle* legacyRect;
32
33 public:
34     RectangleAdapter(LegacyRectangle* rect) : legacyRect(rect) {}
35
36     void draw(int x, int y, int width, int height) override {
37         // Convert new interface to legacy interface
38         int x2 = x + width;
39         int y2 = y + height;
40
41         // Create legacy rectangle with converted parameters
42         LegacyRectangle adaptedRect(x, y, x2, y2);
43         adaptedRect.oldDraw();
44     }
45 }
```

```

44
45     std::cout << "Adapter: Translated new interface to legacy." <<
std::endl;
46 }
47 };
48
49 // Client code
50 int main() {
51     LegacyRectangle* legacy = new LegacyRectangle(0, 0, 20, 30);
52
53     // Using adapter to make legacy compatible with new interface
54     Rectangle* adapter = new RectangleAdapter(legacy);
55     adapter->draw(5, 10, 50, 60); // Uses new interface
56
57     delete legacy;
58     delete adapter;
59
60     return 0;
61 }

```

Listing 3: Adapter Pattern Implementation in C++

4.2 Decorator Pattern

Intent: Attach additional responsibilities to an object dynamically.

Motivation: Decorator provides a flexible alternative to subclassing for extending functionality.

Structure: Component interface $\rightarrow$ ConcreteComponent $\rightarrow$ Decorator $\rightarrow$ ConcreteDecorator.

```

1 #include <iostream>
2 #include <string>
3
4 // Component interface
5 class Coffee {
6 public:
7     virtual std::string getDescription() const = 0;
8     virtual double cost() const = 0;
9     virtual ~Coffee() {}
10 };
11
12 // Concrete Component
13 class SimpleCoffee : public Coffee {
14 public:
15     std::string getDescription() const override {
16         return "Simple Coffee";
17     }
18
19     double cost() const override {

```

```
20         return 2.0;
21     }
22 };
23
24 // Decorator base class
25 class CoffeeDecorator : public Coffee {
26 protected:
27     Coffee* coffee;
28
29 public:
30     CoffeeDecorator(Coffee* c) : coffee(c) {}
31
32     std::string getDescription() const override {
33         return coffee->getDescription();
34     }
35
36     double cost() const override {
37         return coffee->cost();
38     }
39 };
40
41 // Concrete Decorators
42 class MilkDecorator : public CoffeeDecorator {
43 public:
44     MilkDecorator(Coffee* c) : CoffeeDecorator(c) {}
45
46     std::string getDescription() const override {
47         return coffee->getDescription() + ", Milk";
48     }
49
50     double cost() const override {
51         return coffee->cost() + 0.5;
52     }
53 };
54
55 class SugarDecorator : public CoffeeDecorator {
56 public:
57     SugarDecorator(Coffee* c) : CoffeeDecorator(c) {}
58
59     std::string getDescription() const override {
60         return coffee->getDescription() + ", Sugar";
61     }
62
63     double cost() const override {
64         return coffee->cost() + 0.2;
65     }
66 };
67
68 // Client code
69 int main() {
```

```
70 // Start with simple coffee
71 Coffee* coffee = new SimpleCoffee();
72 std::cout << "Order: " << coffee->getDescription()
73           << " Cost: $" << coffee->cost() << std::endl;
74
75 // Add milk
76 coffee = new MilkDecorator(coffee);
77 std::cout << "Order: " << coffee->getDescription()
78           << " Cost: $" << coffee->cost() << std::endl;
79
80 // Add sugar
81 coffee = new SugarDecorator(coffee);
82 std::cout << "Final Order: " << coffee->getDescription()
83           << " Cost: $" << coffee->cost() << std::endl;
84
85 delete coffee;
86 return 0;
87 }
```

Listing 4: Decorator Pattern Implementation in C++

5 Behavioral Design Patterns

Behavioral patterns are concerned with algorithms and the assignment of responsibilities between objects.

5.1 Observer Pattern

Intent: Define a one-to-many dependency so that when one object changes state, all its dependents are notified automatically.

Motivation: Useful in event-driven systems where multiple components must react to changes in another component without being tightly coupled.

Structure: Subject interface $\rightarrow$ ConcreteSubject $\leftrightarrow$ Observer interface $\rightarrow$ ConcreteObserver.

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <algorithm>
5
6 // Forward declaration
7 class Observer;
8
9 // Subject interface
10 class Subject {
11 public:
12     virtual void attach(Observer* observer) = 0;
```

```
13     virtual void detach(Observer* observer) = 0;
14     virtual void notify() = 0;
15     virtual ~Subject() {}
16 };
17
18 // Observer interface
19 class Observer {
20 public:
21     virtual void update(const std::string& message) = 0;
22     virtual ~Observer() {}
23 };
24
25 // Concrete Subject
26 class NewsPublisher : public Subject {
27 private:
28     std::vector<Observer*> observers;
29     std::string latestNews;
30
31 public:
32     void attach(Observer* observer) override {
33         observers.push_back(observer);
34         std::cout << "Observer attached." << std::endl;
35     }
36
37     void detach(Observer* observer) override {
38         observers.erase(std::remove(observers.begin(), observers.end(),
39                                     observer),
40                         observers.end());
41     }
42
43     void notify() override {
44         std::cout << "Notifying " << observers.size() << " observers..."
45         << std::endl;
46         for (Observer* observer : observers) {
47             observer->update(latestNews);
48         }
49
50     void publishNews(const std::string& news) {
51         latestNews = news;
52         std::cout << "\n=== Publishing News: " << news << " ===" << std
53         ::endl;
54         notify();
55     }
56 };
57
58 // Concrete Observer
59 class NewsSubscriber : public Observer {
```

```
60     std::string name;
61
62 public:
63     NewsSubscriber(const std::string& name) : name(name) {}
64
65     void update(const std::string& message) override {
66         std::cout << "Subscriber [" << name << "] received: " <<
message << std::endl;
67     }
68 };
69
70 // Client code
71 int main() {
72     // Create publisher
73     NewsPublisher publisher;
74
75     // Create subscribers
76     NewsSubscriber subscriber1("Alice");
77     NewsSubscriber subscriber2("Bob");
78     NewsSubscriber subscriber3("Charlie");
79
80     // Attach subscribers
81     publisher.attach(&subscriber1);
82     publisher.attach(&subscriber2);
83     publisher.attach(&subscriber3);
84
85     // Publish news (observers get notified)
86     publisher.publishNews("Breaking: New Design Pattern Course
Available!");
87     publisher.publishNews("Update: SOLID Principles Workshop Tomorrow")
;
88
89     // Detach one subscriber
90     publisher.detach(&subscriber2);
91
92     // Publish more news
93     publisher.publishNews("Reminder: Project Submission Deadline
Extended");
94
95     return 0;
96 }
```

Listing 5: Observer Pattern Implementation in C++

6 SOLID Principles

The SOLID principles are five design principles that help create maintainable, understandable, and flexible software. They were introduced by Robert C. Martin (Uncle Bob).

Letter	Principle	Core Idea
S	Single Responsibility	A class should have only one reason to change
O	Open/Closed	Software entities should be open for extension but closed for modification
L	Liskov Substitution	Objects of a superclass should be replaceable with objects of a subclass
I	Interface Segregation	Clients should not be forced to depend on interfaces they don't use
D	Dependency Inversion	Depend on abstractions, not on concretions

Table 3: SOLID Principles Overview

6.1 Single Responsibility Principle (SRP)

Definition: A class should have only one reason to change, meaning it should have only one job or responsibility.

```

1 #include <iostream>
2 #include <string>
3 #include <fstream>
4
5 // BAD DESIGN: Class with multiple responsibilities
6 class EmployeeBad {
7 private:
8     std::string name;
9     std::string position;
10    double salary;
11
12 public:
13     EmployeeBad(const std::string& n, const std::string& p, double s)
14         : name(n), position(p), salary(s) {}
15
16     // Responsibility 1: Employee data management
17     void setSalary(double s) { salary = s; }
18     double getSalary() const { return salary; }
19
20     // Responsibility 2: Report generation (should be separate)
21     void generateReport() {
22         std::cout << "Report for " << name << std::endl;
23         std::cout << "Position: " << position << std::endl;
24         std::cout << "Salary: $" << salary << std::endl;
25     }
26
27     // Responsibility 3: Data persistence (should be separate)
28     void saveToFile(const std::string& filename) {
29         std::ofstream file(filename);
30         file << name << "," << position << "," << salary;
31         file.close();
32     }

```

```
33 };
34
35 // GOOD DESIGN: Separated responsibilities
36 class Employee {
37 private:
38     std::string name;
39     std::string position;
40     double salary;
41
42 public:
43     Employee(const std::string& n, const std::string& p, double s)
44         : name(n), position(p), salary(s) {}
45
46     // Only employee-related methods
47     void setSalary(double s) { salary = s; }
48     double getSalary() const { return salary; }
49     std::string getName() const { return name; }
50     std::string getPosition() const { return position; }
51 };
52
53 // Separate class for report generation
54 class ReportGenerator {
55 public:
56     static void generateEmployeeReport(const Employee& emp) {
57         std::cout << "\n=== Employee Report ===" << std::endl;
58         std::cout << "Name: " << emp.getName() << std::endl;
59         std::cout << "Position: " << emp.getPosition() << std::endl;
60         std::cout << "Salary: $" << emp.getSalary() << std::endl;
61         std::cout << "======" << std::endl;
62     }
63 };
64
65 // Separate class for data persistence
66 class EmployeeRepository {
67 public:
68     static void saveToFile(const Employee& emp, const std::string&
filename) {
69         std::ofstream file(filename);
70         file << emp.getName() << "," << emp.getPosition() << "," << emp
.getSalary();
71         file.close();
72         std::cout << "Employee data saved to " << filename << std::endl
;
73     }
74
75     static Employee loadFromFile(const std::string& filename) {
76         std::ifstream file(filename);
77         std::string name, position;
78         double salary;
79     }
```

```
80         std::getline(file, name, ',');
81         std::getline(file, position, ',');
82         file >> salary;
83
84         return Employee(name, position, salary);
85     }
86 };
87
88 // Client code demonstrating SRP
89 int main() {
90     // Create employee
91     Employee emp("John Doe", "Software Engineer", 75000.0);
92
93     // Generate report using separate class
94     ReportGenerator::generateEmployeeReport(emp);
95
96     // Save data using separate class
97     EmployeeRepository::saveToFile(emp, "employee_data.txt");
98
99     // Load data using separate class
100    Employee loadedEmp = EmployeeRepository::loadFromFile("
employee_data.txt");
101    ReportGenerator::generateEmployeeReport(loadedEmp);
102
103    return 0;
104 }
```

Listing 6: Single Responsibility Principle Example

6.2 Open/Closed Principle (OCP)

Definition: Software entities (classes, modules, functions) should be open for extension but closed for modification.

```
1 #include <iostream>
2 #include <vector>
3 #include <memory>
4
5 // BAD DESIGN: Not following OCP
6 class ShapeBad {
7     // Type field to distinguish shapes
8     enum Type { CIRCLE, SQUARE, TRIANGLE } type;
9
10    // For circle
11    double radius;
12
13    // For square
14    double side;
15
16    // For triangle
```

```
17     double base, height;
18
19 public:
20     ShapeBad(Type t, double r) : type(t), radius(r) {} // Circle
21     ShapeBad(Type t, double s) : type(t), side(s) {} // Square
22     ShapeBad(Type t, double b, double h) : type(t), base(b), height(h)
23     {} // Triangle
24
25     double area() const {
26         switch (type) {
27             case CIRCLE: return 3.14159 * radius * radius;
28             case SQUARE: return side * side;
29             case TRIANGLE: return 0.5 * base * height;
30             default: return 0;
31         }
32     }
33     // Problem: Adding new shape requires modifying this class
34 };
35
36 // GOOD DESIGN: Following OCP
37 class Shape {
38 public:
39     virtual double area() const = 0;
40     virtual ~Shape() {}
41 };
42
43 class Circle : public Shape {
44 private:
45     double radius;
46
47 public:
48     Circle(double r) : radius(r) {}
49
50     double area() const override {
51         return 3.14159 * radius * radius;
52     }
53 };
54
55 class Square : public Shape {
56 private:
57     double side;
58
59 public:
60     Square(double s) : side(s) {}
61
62     double area() const override {
63         return side * side;
64     }
65 };
```

```
66 class Triangle : public Shape {
67 private:
68     double base, height;
69
70 public:
71     Triangle(double b, double h) : base(b), height(h) {}
72
73     double area() const override {
74         return 0.5 * base * height;
75     }
76 };
77
78 // NEW SHAPE: Can be added without modifying existing code
79 class Rectangle : public Shape {
80 private:
81     double width, height;
82
83 public:
84     Rectangle(double w, double h) : width(w), height(h) {}
85
86     double area() const override {
87         return width * height;
88     }
89 };
90
91 // AreaCalculator is closed for modification but open for extension
92 class AreaCalculator {
93 public:
94     static double totalArea(const std::vector<std::shared_ptr<Shape>>&
95 shapes) {
96         double total = 0;
97         for (const auto& shape : shapes) {
98             total += shape->area();
99         }
100         return total;
101     };
102
103 // Client code demonstrating OCP
104 int main() {
105     std::vector<std::shared_ptr<Shape>> shapes;
106
107     // Add various shapes
108     shapes.push_back(std::make_shared<Circle>(5.0));
109     shapes.push_back(std::make_shared<Square>(4.0));
110     shapes.push_back(std::make_shared<Triangle>(3.0, 6.0));
111
112     // New shape added without modifying AreaCalculator
113     shapes.push_back(std::make_shared<Rectangle>(4.0, 6.0));
114 }
```

```
115 // Calculate total area
116 double total = AreaCalculator::totalArea(shapes);
117 std::cout << "Total area of all shapes: " << total << std::endl;
118
119 return 0;
120 }
```

Listing 7: Open/Closed Principle Example

6.3 Liskov Substitution Principle (LSP)

Definition: Objects of a superclass should be replaceable with objects of a subclass without affecting the correctness of the program.

```
1 #include <iostream>
2 #include <vector>
3 #include <memory>
4
5 // BAD DESIGN: Violating LSP
6 class RectangleBad {
7 protected:
8     double width, height;
9
10 public:
11     RectangleBad(double w, double h) : width(w), height(h) {}
12
13     virtual void setWidth(double w) { width = w; }
14     virtual void setHeight(double h) { height = h; }
15
16     double getWidth() const { return width; }
17     double getHeight() const { return height; }
18
19     double area() const { return width * height; }
20 };
21
22 class SquareBad : public RectangleBad {
23 public:
24     SquareBad(double side) : RectangleBad(side, side) {}
25
26     void setWidth(double w) override {
27         width = w;
28         height = w; // Violates LSP: Changes height too
29     }
30
31     void setHeight(double h) override {
32         height = h;
33         width = h; // Violates LSP: Changes width too
34     }
35 };
36
```

```
37 // Problematic usage
38 void testRectangleBad(RectangleBad& rect) {
39     rect.setWidth(5);
40     rect.setHeight(4);
41     std::cout << "Expected area: 20, Actual area: " << rect.area() <<
42     std::endl;
43     // For SquareBad, this prints 16, not 20!
44 }
45 // GOOD DESIGN: Following LSP
46 class ShapeLSP {
47 public:
48     virtual double area() const = 0;
49     virtual ~ShapeLSP() {}
50 };
51
52 class RectangleLSP : public ShapeLSP {
53 protected:
54     double width, height;
55
56 public:
57     RectangleLSP(double w, double h) : width(w), height(h) {}
58
59     virtual void setWidth(double w) { width = w; }
60     virtual void setHeight(double h) { height = h; }
61
62     double getWidth() const { return width; }
63     double getHeight() const { return height; }
64
65     double area() const override { return width * height; }
66 };
67
68 class SquareLSP : public ShapeLSP {
69 private:
70     double side;
71
72 public:
73     SquareLSP(double s) : side(s) {}
74
75     void setSide(double s) { side = s; }
76     double getSide() const { return side; }
77
78     double area() const override { return side * side; }
79 };
80
81 // Factory functions to create shapes
82 std::shared_ptr<RectangleLSP> createRectangle(double width, double
83     height) {
84     return std::make_shared<RectangleLSP>(width, height);
85 }
```

```

85
86 std::shared_ptr<SquareLSP> createSquare(double side) {
87     return std::make_shared<SquareLSP>(side);
88 }
89
90 // Client code demonstrating LSP
91 int main() {
92     std::vector<std::shared_ptr<ShapeLSP>> shapes;
93
94     // Add different shapes
95     shapes.push_back(createRectangle(5, 4));
96     shapes.push_back(createSquare(5));
97
98     // Calculate total area
99     double totalArea = 0;
100     for (const auto& shape : shapes) {
101         totalArea += shape->area();
102         std::cout << "Shape area: " << shape->area() << std::endl;
103     }
104
105     std::cout << "\nTotal area: " << totalArea << std::endl;
106
107     // Demonstrate that each shape maintains its own invariants
108     auto rect = createRectangle(10, 5);
109     std::cout << "\nRectangle area: " << rect->area() << std::endl;
110
111     auto square = createSquare(10);
112     std::cout << "Square area: " << square->area() << std::endl;
113
114     return 0;
115 }

```

Listing 8: Liskov Substitution Principle Example

7 Conclusion

Design Patterns and SOLID Principles are essential tools for professional software development. When applied correctly, they lead to systems that are easier to understand, maintain, test, and extend.

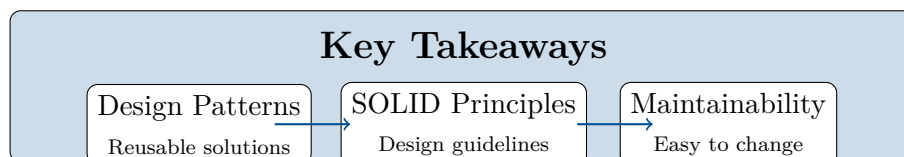


Figure 2: Design Patterns and SOLID Principles Synergy

- **Design Patterns** provide **templates** for solving common problems
- **SOLID Principles** provide **guidelines** for creating maintainable code
- Together, they form a **powerful toolkit** for software architects and developers
- Proper application leads to **reduced technical debt** and **easier maintenance**

Note: Remember that patterns and principles should be applied thoughtfully, not dogmatically. The goal is to create software that is easy to change and maintain, not to use every pattern possible.

FURTHER READING:

- Gamma, E., et al. (1994). *Design Patterns: Elements of Reusable Object-Oriented Software*
- Martin, R. C. (2002). *Agile Software Development: Principles, Patterns, and Practices*
- Freeman, E., et al. (2004). *Head First Design Patterns*